

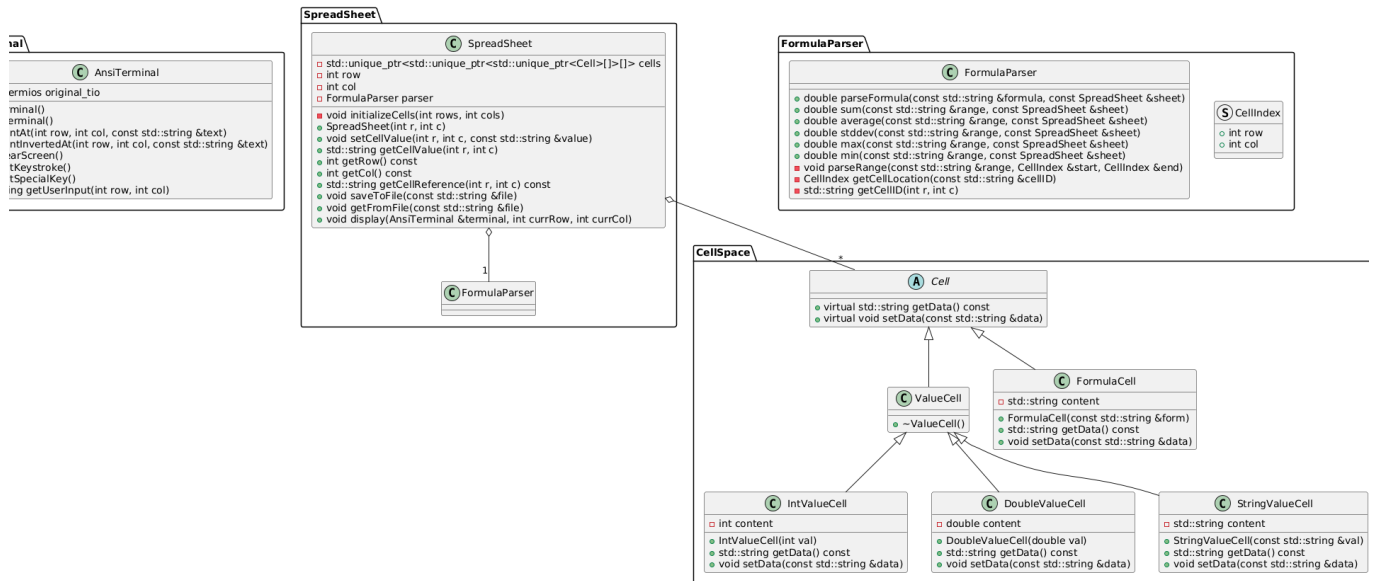


CSE 241 – OBJECT ORIENTED PROGRAMMING

HW#2

Zeynep Etik

Student ID 220104004035



In this assignment there are three main classes. Abstract Cell class, FormulaParser class and SpreadSheet class. Abstract Cell class has two derived classes one for to store formules and named as FormulaCell. Other class is named ValueCell, it can store different kinds of contents. To do that it has three derived classes IntValueCell, DoubleValueCell and StringValueCell classes.

Cell Class

In the Cell class I used namespace to gather all derived classes. And in this class there is just functions to set and get contents. All classes are derived from abstract Cell class.

SpreadSheet Class

In the SpreadSheet class, in the private part there is a 2D unique_ptr to store contents of grid or cell (I will call them as cell). And to prepare the unique_ptr for 2D there is a function called initiliazеCells(). This method creates a kind of 2D array by initiliazеing all the cells by using StringValueCell's constructor. This function is called by SpreadSheet class's

constructor in the start. In the public part, there is a constructor obviously that gets row and col values as parameter.

And some getters such as `getRow()`, `getCol()` and `getCellValue()` methods. `getCellValue()` method firstly controls the type of the class by using downcasting and if it is a formula then calls the `FormulaParser`'s function by using `parser` member variable of `SpreadSheet` to parse the formula and returns the content of the cell as a string.

`void SpreadSheet::setCellValue(int r, int c, const string& input)`

`setCellValue` is one of the most important methods of this class based on the string parameter it decides on which type of cell that input is (is it a member of `StringValueCell` etc.).

To get cell IDs or references such as `A1` of cells there is `getCellReference()`.

`void SpreadSheet::saveToFile(const std::string& file)`

To store the cells into a file there is `saveToFile()` method. It will get through the cells one by one and store them into a file. To be able to store doubles with two precision I controlled the type of the cells by using `dynamic_cast`.

`void SpreadSheet::getFromFile(const std::string& file)`

To get a grid from a file there is `getFromFile()` method. This method will read the file line by line and in every line it will control the commas if there is a comma it will store the data into a temporary vector. And reinitialize the 2D `unique_ptr`

according to new information and set all the values again according to the file's content.

```
void Spreadsheet::display(AnsiTerminal &terminal, int  
currRow, int currCol) const
```

To display the grid I created a method named display(). It takes current row and column values and uses printAt and printInvertedAt according to these values. At the beginning it will print a kind of menu then it will go through all the cells and print them one by one. And each time it is called will clear the terminal by using clearScreen() method from the given AnsiTerminal class.

FormulaParser Class

This class uses a struct named CellIndex to store col and row values of cells accurately. In the private part there are getCellID() method which returns the reference of the class such as A1 and getCellLocation which returns col and row values based on the reference of the class. And lastly to decide on the range of the operations there is parseRange() method. According to the position of this character '..'.

In the public part there are functions to do operations such as =A1*C5 or SUM(B1..B5). And to parse the formulas there is this method:

```
double parseFormula(const string& Formula, const  
SpreadSheet& sheet) const
```

This function finds keywords or key symbols such as AVER or = and according to keywords calculate the value or call the responding method to calculate the formula.

Main Function

In the main() function firstly it asks for desired row and col values and create a SpreadSheet object named as sheet according to the desired row and colmuns. Then starts a while loop by using boolean variable. It uses up, down, left and right arrows to move in the table. According to users command if user wants to edit the table it will get users input and call setCellValue() function. For file operations it will get the file name and call the save or load functions. And after every operstion it will call display() function from SpreadSheet class. And if users want to quit it will change the boolean's value. At the end of the program screen will be cleared and there will be goodbye message.

Missing Parts Of The Program

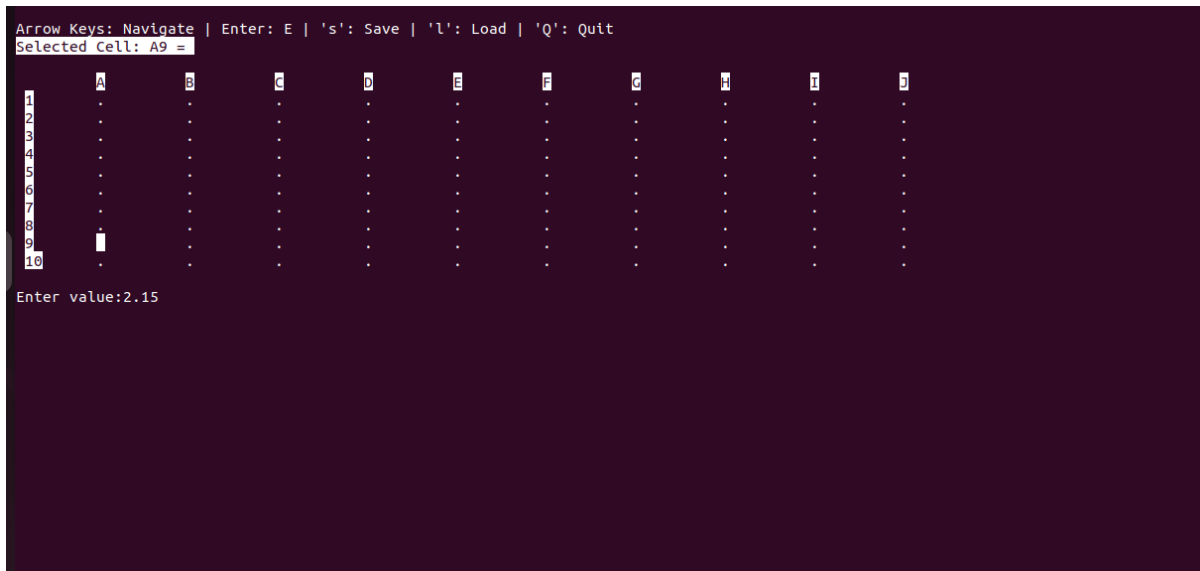
Grid does not go through the terminal which means the maximum width will be the end of the grid. And since I could not figure out where to use template types I did not use them.

Parts Which Are AI Assisted

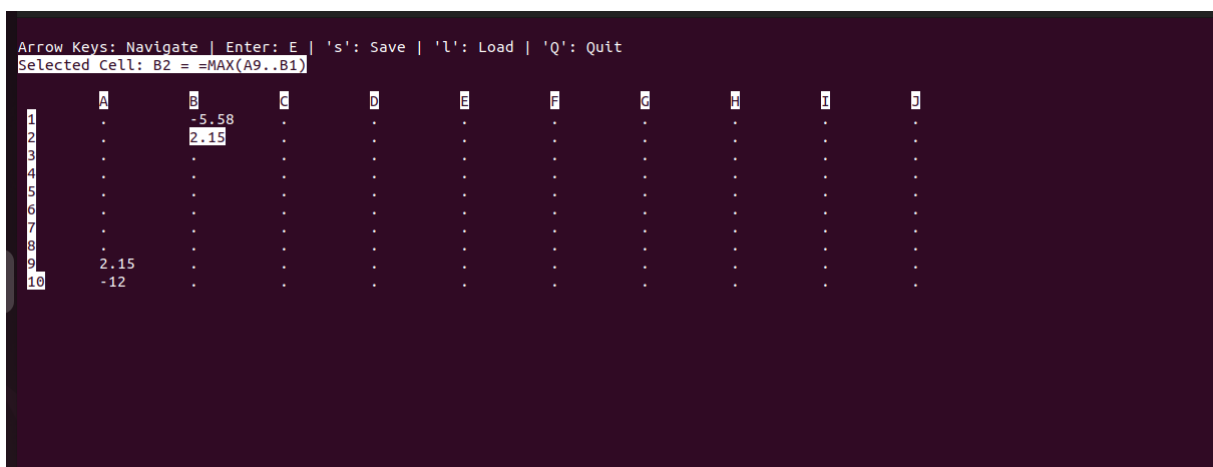
To figure out how to use 2D unique_ptr I used AI assistance. Also in the some parts of the program I had encountered some problems so I just asked the AI.

User Guide

User can move by using right, left, up and down arrows of the keyboard. If they want to edit a specific cell they should enter 'E' character.



If they want to add an formula (such as A1+14, SUM(B1..B4), etc.) they should insert '=' character then write the formula. Formulas should be written in capital letters. Formats of the formulas should be as shown below: =MAX(A10..B1)



=STDDEV(B2..B4)

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: C5 = =STDDEV(B2..B4)

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

=MIN(B2..B4)

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: C7 = =MIN(B2..B4)

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

=B2/B4

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: D5 = =B2/B4

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

=AVER(B2..B4)

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: F4 = =AVER(B2..B4)

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

=SUM(B1..B4)

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: A10 = =SUM(B1..B4)

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

To save the grid into a file user should press 's'. And enter the file name.

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: A10 = =SUM(B1..B4)

	A	B	C	D	E	F	G	H	I	J
1	.	-3.15	.	.	.	.	.	.	.	.
2	.	15.52	.	.	.	.	.	.	.	.
3	.	21	.	.	.	.	.	.	.	.
4	.	-12	.	.	.	8.17	.	.	.	.
5	.	.	14.44	-1.29	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.	.	.
7	.	.	-12.00	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.	.	.
9	.	21.37	.	.	.	.	.	.	.	.
10	21.37	.	.	.	.	.	.	.	.	.

Enter file name to save:
Enter value:HEY

And the file that grid has been stored is like that:

FormulaParser.cpp	×	HEY	×	HiH	×	hey
-------------------	---	-----	---	-----	---	-----

```

1 ,-3.15,,,,,,,,
2 ,15.52,,,,,,,,
3 ,21,,,,,,,,
4 ,-12,,,=AVER(B2..B4),,,,
5 ,,,=STDDEV(B2..B4),=B2/B4,,,,,
6 ,,,,,,,,,
7 ,,,=MIN(B2..B4),,,,,,
8 ,,,,,,,,,
9 ,=MAX(A10..B4),,,,,,
10 =SUM(B1..B4),,,,,,,,,

```

And to get grid from a file. User should press 'l'. And write the file name as shown below:

```

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: A1 =

1      A

Enter file name to load:
Enter value:HiH

```

And file was like that:

```

FormulaParser.cpp  x  HEY  x  HiH  x
1 ,,,,,,,,,,
2 ,15.52,,,,,,,,
3 ,42,,,,,,,,,
4 ,2.02,,,,,,,,,
5 ,,,=B2/B4,,,,,,,,
6 ,,,,,,,,,,
7 ,,,=MIN(B2..B4),,,,,,,,,
8 ,,,,,,,,,,
9 ,,,,,,,,,,
10 ,,,,,,,,,,

```

And here is how grid is shown after loading file:

```

Arrow Keys: Navigate | Enter: E | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: A1 =

1      A      B      C      D      E      F      G      H      I      J
2      .      15.52  .      .      .      .      .      .      .
3      .      42     .      .      .      .      .      .      .
4      .      2.02   .      .      .      .      .      .      .
5      .      .      .      7.68  .      .      .      .      .
6      .      .      .      .      .      .      .      .      .
7      .      .      2.02  .      .      .      .      .      .
8      .      .      .      .      .      .      .      .      .
9      .      .      .      .      .      .      .      .      .
10     .      .      .      .      .      .      .      .      .

```